Università degli Studi di Salerno

Corso di Ingegneria del Software

TrackFit
System Design Document
Versione 1.1

TrackFit
Sistema di Tracking
Allenamenti

Revision History

Coordinatore del progetto:

Nome	**Matricola**
Gabriele di Palma	0512119257

Partecipanti:

Nome	**Matricola**
Yuri Fierro	0512119524

Scritto da:	Gabriele di Palma

Revision History

Data	**Versione**	**Descrizione**	**Autore**
26/02/2025	1.0	Prima Stesura SDD	GDP, YF
02/03/2025	1.1	Modifica architettura del sistema proposto	GDP,YF

Sommario

Revision History....2
1. Introduzione....4
1.2 Design Goals....4
1.3 Trade-Offs....6
1.4 Definizioni, acronimi e abbreviazioni....6
1.5 Panoramica....7
2. Architettura del sistema corrente....7
3. Architettura del sistema proposto....8
3.1 Panoramica....8
3.2 Decomposizione in Sottosistemi....8
3.3 Mapping Hardware / Software....9
3.4 Gestione Dati Persistenti....10
3.5 Controllo degli accessi e sicurezza....11
3.6 Controllo del Flusso Globale....12
3.7 Condizioni Boundary....13
4. Servizi dei Sottosistemi....13
5. Glossario....16

1. Introduzione

In questo documento descriveremo come si intende procedere con l'architettura del sistema TrackFit e quali sono gli obiettivi di design che intendiamo raggiungere.

1.1 Scopo del sistema

TrackFit si propone come una piattaforma digitale per il monitoraggio e la gestione delle attività sportive, rivolta a utenti amatoriali e semi-professionisti. La piattaforma consente la registrazione e l'analisi delle sessioni di allenamento, la gestione delle palestre e degli abbonamenti, il monitoraggio delle performance e la generazione di reportistica personalizzata.

Il sistema è accessibile tramite interfaccia web (Flutter) e applicazione mobile , e gestisce quattro categorie principali di attori:

- **Utente/Atleta:** registra allenamenti, prenota corsi, gestisce il proprio abbonamento.
- **Coach/Allenatore:** pianifica e modifica i piani di allenamento degli atleti associati.
- **Gestore della palestra:** amministra struttura, corsi, orari e gestisce i report gestionali.
- **Amministratore:** supervisiona tutte le strutture connesse al sistema con visione aggregata.

1.2 Design Goals

Gli utenti principali di TrackFit sono gli atleti e i gestori di palestre. È necessario fornire un'esperienza d'uso semplice e intuitiva, garantendo al contempo affidabilità nella gestione di dati sensibili (parametri fisici, pagamenti) e scalabilità per gestire picchi di carico durante eventi sportivi. In base a queste osservazioni, si identificano i seguenti obiettivi di design:

- **Disponibilità:** il sistema deve essere disponibile h24, 7 giorni su 7, con un uptime garantito del 99,5%. Gli atleti devono poter registrare un allenamento o prenotare un corso in qualsiasi momento, indipendentemente dal fuso orario.
- **Scalabilità:** il sistema deve supportare almeno 1000 utenti concorrenti mantenendo i tempi di risposta sotto i 2 secondi (NFR-P1, NFR-P3). L'architettura deve permettere la scalabilità orizzontale per gestire picchi di carico, ad esempio durante eventi sportivi di massa (scenario maratona, UC5).
- **Sicurezza:** TrackFit gestisce dati personali sensibili (parametri fisici, documenti), dati finanziari (pagamenti, abbonamenti) e credenziali di accesso. Il sistema deve essere conforme al GDPR (NFR-L1) e agli standard PCI-DSS

(NFR-L2). Tutte le comunicazioni devono avvenire tramite HTTPS e le password devono essere cifrate con bcrypt (NFR-R3, NFR-R5).

- **Gestione dei malfunzionamenti:** dovendo gestire sessioni di allenamento in corso e transazioni economiche, i malfunzionamenti devono essere gestiti garantendo l'integrità dei dati. Particolare attenzione va al rispetto delle proprietà ACID nelle transazioni di pagamento e al recupero delle sessioni interrotte (scenario crash batteria, UC4 alternativa).
- **Performance:** le operazioni standard (login, visualizzazione dashboard, apertura pagine) devono essere completate in meno di 2 secondi (NFR-P1). La generazione di report complessi deve essere completata in meno di 5 secondi (NFR-P2). Il sistema deve implementare strategie di caching per le query frequenti.

Rank/Priorità	ID	Design Goal	Categoria	Trade-off
1	DG_1	Sicurezza dei dati utente tramite autenticazione	Dependability	Performance vs Security
2	DG_2	Elevata scalabilità del sistema cloud	Performance	Cost vs Reliability
3	DG_3	Accesso multi-ruolo (utente, coach, palestra, admin)	Maintenance	Complexity vs Flexibility
4	DG_4	Interfaccia user-friendly	End-user	Time vs Quality
5	DG_5	Disponibilità continua del servizio	Dependability	Cost vs Availability
6	DG_6	Aggiornamento realtime delle statistiche	Performance	Network usage
7	DG_7	Manutenibilità del codice	Maintenance	Release Time vs Quality

8	DG_8	Compatibilità browser multipiattaforma	Performance	Optimization vs Compatibility

1.3 Trade-Offs

Trade-Offs	Descrizione
Sicurezza vs Prestazioni	L'utilizzo di autenticazione Firebase aumenta la sicurezza ma introduce latenze
Scalabilità vs Costi	L'infrastruttura cloud comporta costi maggiori
Tempo di rilascio vs Qualità	Alcune funzionalità avanzate potranno essere introdotte in release successive
Compatibilità vs Prestazioni	Supportare più browser riduce ottimizzazioni specifiche

1.4 Definizioni, acronimi e abbreviazioni

Acronimi:

• RAD = Requirement Analysis Document
• SDD = System Design Document
• COTS = Commercial Off-the-Shelf

Definizioni:
• Sottosistema: corrisponde alla parte di lavoro che può essere svolta autonomamente da un singolo sviluppatore o da un gruppo di sviluppatori. Si ottiene decomponendo il sistema.
• **Design Goal**: proprietà del sistema sulla quale ci si concentra maggiormente.
• **Trade-off:** possibilità di ridurre una certa qualità per aumentare il valore di un'altra qualità e viceversa. Il termine è espresso talvolta come costo opportunità, riferendosi a più alternative alle quali si è preferito rinunciare a vantaggio di un'altra scelta.
• **Mapping Hardware-Software**: descritto per indicare i vari device hardware utilizzati dal sistema e la loro interazione con le componenti software.
• **Dati Persistenti**: dati che devono sopravvivere all'esecuzione singola dell'applicazione. Sono i dati che vengono salvati nel database.

1.5 Panoramica

Il presente documento è strutturato in quattro sezioni principali:

• **Introduzione**: in questa sezione del documento è possibile trovare una descrizione dello scopo del sistema, i vari design goals ordinati per priorità ed arricchiti di descrizione, i trade-offs ed informazioni circa il linguaggio ed i riferimenti utilizzati.

• **Architettura del sistema corrente**: in questa sezione del documento è possibile analizzare l'architettura del sistema attualmente esistente.

• **Architettura del sistema proposto**: in questa sezione del documento è possibile analizzare tramite una descrizione accurata l'architettura del sistema da noi proposto.

Tale sezione è a sua volta suddivisa in:
• decomposizione dei sottosistemi;
• mapping hardware/software;
• gestione dati persistenti;
• controllo degli accessi e sicurezza, del flusso globale del sistema;

• **Servizi dei sottosistemi**: in questa sezione del documento è possibile leggere la descrizione dei servizi di ogni sottosistema proposto.
• **Glossario**: sezione del documento che associa ad ogni sigla/termine menzionato/a all'interno del documento una definizione/descrizione per evitare che il lettore vada a ricercare informazioni all'esterno del documento, rendendolo autosufficiente sulla materia trattata.

2. Architettura del sistema corrente

Attualmente non esiste un sistema unificato che consenta simultaneamente:

- monitoraggio degli atleti,
- gestione degli allenatori,
- analisi statistiche della palestra,
- comunicazione tra utenti sportivi.

Le soluzioni esistenti gestiscono solo singoli aspetti (fitness tracking personale oppure gestione amministrativa delle palestre), senza integrare tutte le funzionalità in un'unica piattaforma.

TrackFit nasce quindi come sistema integrato per la gestione completa dell'ecosistema sportivo.

3. Architettura del sistema proposto

3.1 Panoramica

Nelle prossime sezioni andremo a definire l'architettura del sistema in termini di:

- **Decomposizione in sottosistemi:** si propone un'architettura a tre livelli con suddivisione tra interfacce utente, logica applicativa e logica di persistenza.
- **Hardware/software mapping:** distribuzione dei componenti software sui dispositivi fisici (client, server cloud, database).
- **Gestione dei dati persistenti:** struttura e tecnologie utilizzate per il salvataggio dei dati.
- **Controllo di accessi e sicurezza:** tabella degli accessi per ruolo e meccanismi di autenticazione/autorizzazione.
- **Controllo globale del software:** approccio al control flow e gestione della concorrenza.
- **Boundary conditions:** gestione dei casi limite e degli scenari di errore.

3.2 Decomposizione in Sottosistemi

Per la divisione in sottosistemi abbiamo analizzato i requisiti funzionali e non funzionali del RAD. Considerando la natura multi-ruolo e multi-piattaforma (web + mobile) di TrackFit, abbiamo optato per un'architettura a tre livelli:

1. Le interfacce degli utenti (divise per ruolo e per piattaforma: Web e Mobile)
2. La logica applicativa (sottosistemi per dominio funzionale)
3. La logica di persistenza (Storage)

Le interfacce utente comunicano tramite API RESTful con i sottosistemi del livello applicativo. I permessi di accesso sono descritti nella tabella degli accessi (sez. 3.5).

I sottosistemi della logica applicativa individuati sono:

- **UserManagement:** gestisce registrazione, autenticazione, profili e ruoli (FR1–FR4, FR29–FR30).
- **GymManagement:** gestisce palestre, corsi, orari e capacità (FR5–FR8).
- **WorkoutTracking:** gestisce la pianificazione, l'esecuzione e il consolidamento delle sessioni di allenamento (FR9–FR11, FR23–FR25).

- **ReportingManagement:** gestisce la generazione di report e dashboard per tutti i ruoli (FR12–FR15, FR28).
- **PaymentManagement:** gestisce abbonamenti, pagamenti, fatturazione e coupon (FR7, FR20–FR22).
- **InteroperabilityManagement:** gestisce l'importazione/esportazione dati in formati standard (FR27–FR28).

I sottosistemi della logica applicativa comunicano con il componente Storage per il salvataggio persistente dei dati. Componenti allo stesso livello possono comunicare tra loro dove necessario (es. WorkoutTracking con ReportingManagement).

Di seguito il Component Diagram del sistema:

```
── Interface ──────────────────────────────────────────────────────────
    <<component>> WebInterface    <<component>> MobileInterface
── ApplicationLogic ───────────────────────────────────────────────────
        <<component>>      <<component>>      <<component>>
         UserManagement   GymManagement   WorkoutTracking
       <<component>>      <<component>>      <<component>>
   ReportingManagement  PaymentManagement  InteroperabilityMgmt
── Storage ────────────────────────────────────────────────────────────
                    <<component>> Storage (Firebase)
```

3.3 Mapping Hardware / Software

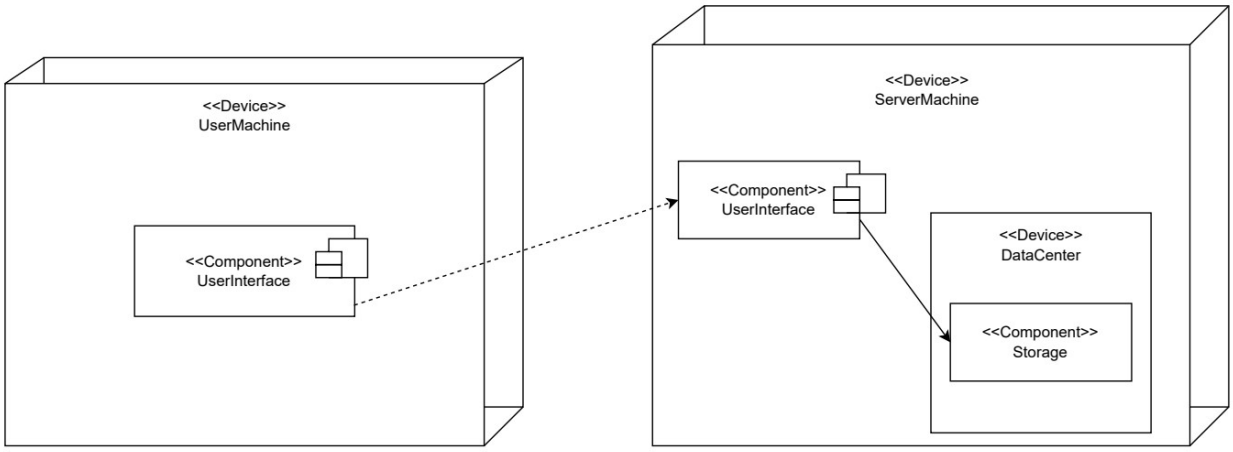
TrackFit nasce come sistema distribuito cloud-native, in cui client web e mobile da qualsiasi dispositivo si connettono a server centralizzati che gestiscono le richieste tramite API RESTful. È compito del server verificare i permessi dell'utente autenticato prima di eseguire qualsiasi operazione (NFR-R4).

Il sistema sarà sviluppato utilizzando il framework Flutter , che facilita la creazione di applicazioni web server-side rendering (SSR) e static generation, riducendo al minimo la necessità di configurazioni manuali. Flutter è ideale per implementare sia il frontend che le API Routes del backend, offrendo una gestione efficiente del routing e delle dipendenze. Per garantire un controllo sicuro degli accessi e la gestione dei token di sessione, verrà utilizzato JWT (JSON Web Token) integrato con middleware Next.js. Sul lato mobile verrà impiegato React Native, che permette di sviluppare applicazioni native iOS e Android condividendo la logica applicativa con il frontend web.

La distribuzione avverrà su un provider Cloud (es. AWS o Vercel) con le seguenti componenti hardware/software:

Nodo	Tipo	Componenti Software
UserMachine (Web)	<<device>> Browser	WebInterface (Next.js / React)
UserMachine (Mobile)	<<device>> Smartphone (iOS/Android)	MobileInterface (React Native / PWA)
ServerMachine	<<device>> Cloud Server (AWS/Vercel)	ApplicationLogic (Next.js API Routes / Node.js)
DatabaseMachine	<<device>> Cloud DB (AWS RDS)	Storage (MySQL + Redis Cache)

UserInterface e ApplicationLogic comunicano tramite chiamate HTTPS alle API RESTful. Il livello Storage è accessibile esclusivamente da ApplicationLogic e non è mai esposto direttamente ai client.



3.4 Gestione Dati Persistenti

Il sistema dovrà gestire il salvataggio persistente di:

- I dati degli utenti (profilo, credenziali cifrate, parametri fisici)
- Le sessioni di allenamento (metriche, tracciati GPS, note, tag)
- I piani di allenamento e i calendari degli atleti
- Le strutture, i corsi e gli orari delle palestre
- Gli abbonamenti e lo storico dei pagamenti
- Le fatture, i coupon e i QR code per l'accesso

- I report generati e le impostazioni degli obiettivi
- I log di audit per tutte le operazioni amministrative (R10)
- Le liste d'attesa per i corsi al completo

Per farlo verrà utilizzato un database relazionale MySQL, in modo da garantire accesso veloce ai dati e il rispetto delle proprietà ACID nelle transazioni, in particolare per le operazioni di pagamento e aggiornamento degli abbonamenti (NFR-R2). La comunicazione con il database avverrà tramite un ORM (es. Prisma o TypeORM) per garantire type-safety e migrazioni controllate.

Per migliorare le performance (NFR-P1, NFR-P2) verrà integrato un layer di caching con Redis per le query frequenti (dashboard, elenco corsi, quote disponibili) e per la gestione delle sessioni autenticate.

Verranno eseguiti backup automatici giornalieri del database (NFR-R2), con RPO massimo di 24 ore e RTO massimo di 4 ore in caso di disastro, gestiti tramite i servizi del provider cloud (es. AWS RDS automated backups).

3.5 Controllo degli accessi e sicurezza

La Global Access Table definisce i permessi associati ai vari ruoli all'interno di TrackFit, descrivendo le azioni che ciascun ruolo è autorizzato a compiere sulle entità del sistema. La tabella rappresenta gli attori nelle righe e le entità a cui possono accedere nelle colonne, specificando le operazioni consentite per ciascuna combinazione.

Attori / Oggetti	**Workout**	**Profilo**	**Palestra/ Corso**	**Abbonamento**	**Report**	**Pagamento**	**Utenti (Staff)**
Utente	create read update delete (propri)	read update (proprio)	read prenotare cancellare	read attivare rinnovo	read esportare (propri)	effettuare visualizzare storico	—
Coach	read (atleti associati) proponi	read (atleti assoc.)	read (propri corsi)	read (atleti assoc.)	read generare (atleti assoc	—	—

	Modifica				.)		
Gestore	—	read (iscritti)	create read update delete	create read update configurare	read generare (struttura)	visualizzare (struttura)	associare e rimuovere Coach
Amministratore	—	read (tutti)	create read update delete (tutte)	configurare tutte	read tutte aggregati	visualizzare tutte	create update delete (tutti ruoli)

Il controllo degli accessi sarà affidato al componente UserManagement, che tramite middleware JWT verifica il token di sessione e il ruolo associato all'account prima di consentire l'accesso a qualsiasi endpoint API (NFR-R3, NFR-R4). L'accesso è negato di default e concesso esplicitamente solo per le operazioni autorizzate.

Sicurezza adottata

- Autenticazione Firebase
- Accesso role-based
- Connessioni Https
- Regole Firestore Security rules

3.6 Controllo del Flusso Globale

In TrackFit, il control flow sarà implementato utilizzando un approccio event-driven e request-response, supportato dall'architettura nativa di Flutter. Le richieste generate dai client (HTTP/HTTPS) vengono ricevute dal router di Flutter, che le instrada al corretto API Route Handler. All'interno degli handler, i servizi applicativi necessari vengono invocati per elaborare le azioni richieste, centralizzando l'elaborazione degli input e mantenendo una struttura organizzata a livelli.

3.7 Condizioni Boundary

Le condizioni di confine in TrackFit si riferiscono a tutti i casi limite o alle situazioni estreme che potrebbero influenzare il comportamento del sistema. Il monitoraggio e la gestione di queste condizioni sono affidati a un amministratore di sistema che supervisiona l'infrastruttura cloud, esegue operazioni di manutenzione e interviene in caso di malfunzionamenti.

Le principali condizioni di confine identificate sono:

- **Crash del dispositivo mobile durante il tracciamento:** la sessione viene salvata localmente ogni 30 secondi su file temporaneo. Al riavvio dell'app, il sistema rileva l'anomalia e propone il recupero dei dati (UC4 alternativa, scenario trekking).
- **Assenza di rete durante la sincronizzazione:** il sistema sospende l'upload, imposta lo stato della sessione in 'In attesa di sincronizzazione' e avvia un servizio in background che monitora il ripristino della connessione (UC14).
- **Picchi di carico simultaneo:** l'architettura cloud con auto-scaling e code di messaggi (Redis/Bull) garantisce che migliaia di sessioni simultanee vengano accodate e processate senza perdite di dati (NFR-P3, NFR-P4).
- **Transazione di pagamento fallita:** il sistema annulla immediatamente la risorsa creata (prenotazione o abbonamento), sblocca lo slot e notifica l'utente senza addebiti (UC16).
- **Tentativo di creazione struttura duplicata:** il sistema blocca la scrittura sul DB prima del commit, segnala il campo in conflitto all'amministratore e preserva l'integrità del database (UC13, R8, R9).
- **Violazione del vincolo di preavviso 48h:** il sistema blocca l'operazione del Coach prima della scrittura e offre l'alternativa di aggiungere una nota all'atleta (UC15, R1).
- **Deployment e manutenzione:** il sistema supporta zero-downtime deploy tramite strategie blue-green o rolling update gestite dal provider cloud (NFR-P5).

4. Servizi dei Sottosistemi

Analizziamo ora per i sottosistemi di logica applicativa i servizi che devono offrire, con i relativi metodi esposti.

UserManagement

Sono definiti i servizi Authentication e UserProfile, che raccolgono le funzionalità di gestione degli utenti registrati, dei ruoli e della sicurezza degli account (FR1–FR4, FR29–FR30).

Servizio	Metodi esposti
Authentication	register(email, password, role) login(email, password) → JWT logout(token) resetPassword(email) deleteAccount(userId)
UserProfile	getProfile(userId) updateProfile(userId, data) associateCoach(userId, coachId) getUsersByRole(role)

GymManagement

Sono definiti i servizi GymControl e CourseControl, che raccolgono le funzionalità di creazione e gestione delle strutture, dei corsi e delle prenotazioni (FR5–FR8, FR24–FR26).

Servizio	Metodi esposti
GymControl	createGym(data) – verifica anti-duplicato (R8, R9) getGym(gymId) updateGym(gymId, data) deactivateGym(gymId) assignCoach(gymId, coachEmail)
CourseControl	createCourse(gymId, data) getCourseSchedule(gymId) bookCourse(userId, courseId) – verifica capienza e abbonamento cancelBooking(bookingId) – verifica finestra temporale (R3) addToWaitlist(userId, courseId) notifyWaitlist(courseId) cancelCourse(courseId) – notifica tutti gli iscritti (FR26)

WorkoutTracking

Sono definiti i servizi SessionControl e AccessControl, che raccolgono le funzionalità di pianificazione, esecuzione, consolidamento delle sessioni di allenamento e gestione degli accessi fisici alla struttura (FR9–FR11, FR23–FR25).

Servizio	Metodi esposti
SessionControl	planSession(userId, sessionData) startTracking(sessionId) – attiva GPS/sensori saveLocalSnapshot(sessionData) – salvataggio ogni 30s recoverSession(deviceId) – recupero dopo crash stopTracking(sessionId, metrics) consolidateSession(sessionId) – deduplicazione e conferma updateGoalProgress(userId, sessionId)

AccessControl	generateQRCode(subscriptionId) validateAccess(qrCode, gymId) – verifica abbonamento attivo (R7) recordCheckIn(userId, gymId)

ReportingManagement

È definito il servizio ReportControl, che raccoglie le funzionalità di generazione e download dei report per tutti i ruoli, con accesso differenziato ai dati (FR12–FR15, FR28).

Servizio	Metodi esposti
ReportControl	generateUserReport(userId, filters) → PDF/CSV generateCoachReport(coachId, filters) → PDF generateGymReport(gymId, filters) → PDF/CSV generateAdminReport(filters) → PDF aggregato exportData(userId, format) – GPX, CSV, ZIP

PaymentManagement

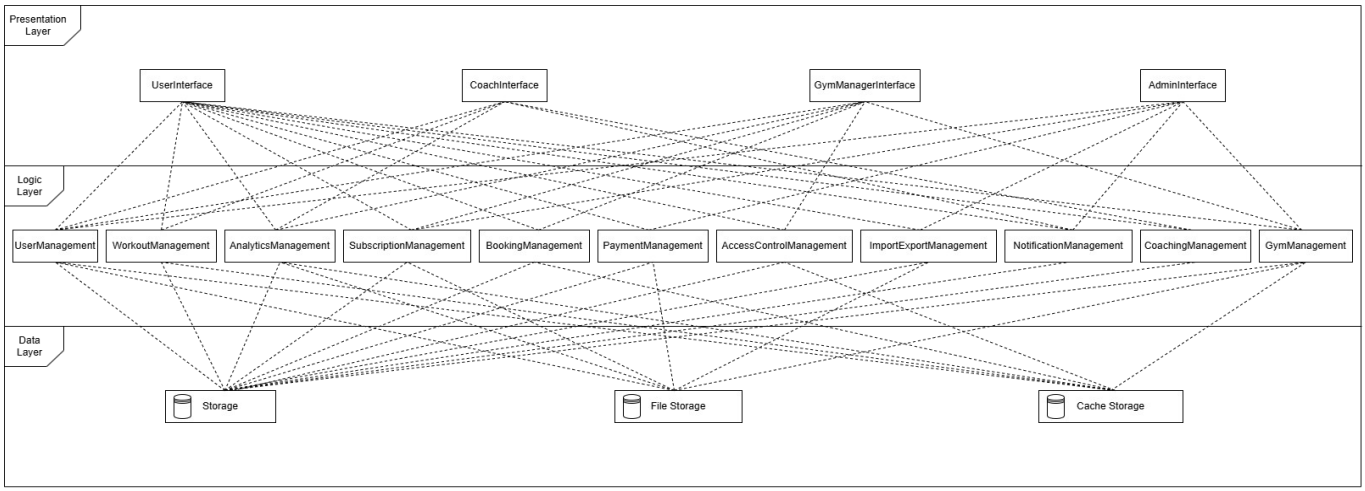
Sono definiti i servizi SubscriptionControl e PaymentControl, che raccolgono le funzionalità di gestione degli abbonamenti, dei coupon, dei pagamenti via gateway esterno (Stripe) e della fatturazione (FR7, FR20–FR22, UC8).

Servizio	Metodi esposti
SubscriptionControl	getPlans(gymId) activateSubscription(userId, planId, couponCode) validateCoupon(code, gymId, planId, userId) – verifica R4, R5 cancelSubscription(subscriptionId) toggleAutoRenew(subscriptionId) getSubscriptionStatus(subscriptionId)
PaymentControl	initiateCheckout(userId, amount) → Stripe session handleWebhook(stripeEvent) – conferma pagamento generateinvoice(paymentId) → PDF getPaymentHistory(userId) refundPayment(paymentId) – in caso di cancellazione (R3)

InteroperabilityManagement

È definito il servizio InteropControl, che raccoglie le funzionalità di importazione da formati standard (GPX, TCX) e di esportazione/condivisione verso API terze (Strava, Google Fit, Instagram) tramite OAuth 2.0 (FR27–FR28).

Servizio	Metodi esposti
InteropControl	importGPX(file) → sessionData importTCX(file) → sessionData importFromStrava(userId, oauthToken) exportToStrava(sessionId, oauthToken) syncGoogleFit(sessionId, oauthToken) generateShareImage(sessionId) → PNG (per Instagram/Social)



5. Glossario

Termine	Definizione
Atleta	Utente che svolge allenamenti
Allenatore	Gestisce atleti
Palestra	Struttura sportiva
Admin	Amministratore sistema
Workout	Programma allenamento
Firebase	Backend cloud